



第18课时：函数进阶——递归入门

自调用 · 边界 · 递推公式

递归的概念、两要素（边界条件 + 递归公式）、经典示例



上节课回顾



上节课我们学习了函数的基础：

- **函数定义：** `返回值类型 函数名(参数) { 函数体; return 返回值; }`
- **函数调用：** 用 `函数名(实参)` 触发执行
- **返回值：** 通过 `return` 返回；`void` 表示无返回值
- **值传递：** 形参是实参的"副本"，函数内修改不影响实参
- **函数声明：** 原型分号结尾，可先声明后定义

本节课我们将学习一种特殊的函数——**递归**。

什么是递归？



递归 = **递**（递进） + **归**（回归）

指**函数调用自身**的编程技巧：

```
int func() {  
    // ...  
    func();    // 函数调用自己  
    // ...  
}
```

听起来很奇怪——自己调用自己，会停下来吗？

💡 **会！** 只要满足**边界条件**，递归就会停下来。



生活中的递归



俄罗斯套娃：

- 打开一个套娃，里面是**更小**的套娃
- 一直打开，直到最小的套娃（**没有**更小的）

镜子照镜子：

- 镜子里有**更小**的自己
- 一直照，直到无穷小（理论）

这些都符合递归结构：问题 = 简化版的同类问题。

递归的两要素



写递归函数必须满足**两要素**：

1. **边界条件（递归出口）**：问题简化到最小时，**直接给出答案**，不再递归
2. **递归公式**：把当前问题转化为**更小的同类问题**

边界条件：f(最小情况) = 已知值

递归公式：f(n) = 用 f(更小的n) 表示 f(n)

⚠ **缺一不可**：缺边界条件 → **死循环**（无限递归，最终栈溢出）。

例题 1：阶乘



题目描述

用递归求 $n!$ (n 的阶乘)。

输入输出示例：

- 输入：5 → 输出：120
- 输入：0 → 输出：1

解题思路

根据阶乘的数学定义直接写递归：

- **边界条件：** $0! = 1$ (最小情况直接返回)
- **递归公式：** $n! = n \times (n-1)!$ (把 $n!$ 转化为更小的 $(n-1)!$)

函数 `factorial(n)` 调用 `factorial(n-1)`，直到 `n == 0` 返回 1，再逐层乘回去。

例题 1：阶乘 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int factorial(int n) {
    if (n == 0) return 1;          // 边界条件
    return n * factorial(n - 1);   // 递归公式
}

int main() {
    cout << factorial(5) << endl; // 120
    return 0;
}
```

阶乘的执行过程



调用 `factorial(5)` 的展开：

```
factorial(5)
= 5 * factorial(4)
  = 4 * factorial(3)
    = 3 * factorial(2)
      = 2 * factorial(1)
        = 1 * factorial(0)
          = 1          ← 边界返回
            = 1 * 1 = 1
              = 2 * 1 = 2
                = 3 * 2 = 6
                  = 4 * 6 = 24
                    = 5 * 24 = 120 ← 最终结果
```

💡 "递"：层层向下调用。"归"：层层向上返回结果。

例题 2：斐波那契数列



题目描述

用递归求斐波那契数列的第 n 项。数列：1, 1, 2, 3, 5, 8, 13, 21, ...

输入输出示例：

- 输入：7 → 输出：13

解题思路

根据斐波那契数列的数学定义：

- **边界条件：** $\text{fib}(1) = 1$, $\text{fib}(2) = 1$ （前两项已知）
- **递归公式：** $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ （第 n 项 = 前两项之和）

⚠ 注意：递归会有**重复计算**（如 $\text{fib}(3)$ 被多次调用）， n 很大时效率低。

例题 2：斐波那契数列 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int fib(int n) {
    if (n == 1 || n == 2) return 1;           // 边界
    return fib(n - 1) + fib(n - 2);         // 递归
}

int main() {
    cout << fib(7) << endl;    // 13
    return 0;
}
```

数列：1, 1, 2, 3, 5, 8, 13, 21, ...

斐波那契的执行过程



调用 `fib(4)` :

```
      fib(4)
     /    \
   fib(3)  fib(2)
  /  \    |
fib(2) fib(1) 1
 |      |
1       1
= 1 + 1 = 2 ← fib(3)
= 2 + 1 = 3 ← fib(4)
```

💡 注意：递归会**重复计算**某些子问题（如 `fib(2)` 被算了 2 次）。`n` 很大时效率低，下节课会用**记忆化**优化。

例题 3：求 $1+2+\dots+n$



题目描述

用递归求 $1 + 2 + \dots + n$ 的和。

输入输出示例：

- 输入：100 → 输出：5050

解题思路

把累加问题转化为递归：

- **边界条件：** $\text{sum}(1) = 1$ （到 1 时直接返回 1）
- **递归公式：** $\text{sum}(n) = \text{sum}(n-1) + n$ （前 $n-1$ 项的和 + 第 n 项）

💡 对比上节课用循环求和的写法，递归写法更接近数学定义。

例题 3：求 $1+2+\dots+n$ —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int sumToN(int n) {
    if (n == 1) return 1;
    return sumToN(n - 1) + n;
}

int main() {
    cout << sumToN(100) << endl;    // 5050
    return 0;
}
```

例题 4：数字之和



题目描述

输入一个正整数 n ，输出它各位数字之和。

输入输出示例：

- 输入：123 → 输出：6 ($1+2+3=6$)
- 输入：9999 → 输出：36 ($9+9+9+9=36$)

解题思路

把"各位数字之和"转化为递归：

- **边界条件：** $n < 10$ 时（个位数），直接返回 n
- **递归公式：** $\text{digitSum}(n) = \text{digitSum}(n/10) + n\%10$
 - $n/10$ 去掉最后一位（缩小问题）
 - $n\%10$ 取出最后一位（当前层的贡献）

执行过程： $\text{digitSum}(123) \rightarrow \text{digitSum}(12) + 3 \rightarrow (\text{digitSum}(1) + 2) + 3 \rightarrow 1 + 2 + 3 = 6$



例题 4：数字之和 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int digitSum(int n) {
    if (n < 10) return n;          // 个位数直接返回
    return digitSum(n / 10) + n % 10;
}

int main() {
    cout << digitSum(123) << endl; // 6
    return 0;
}
```

例题 5：求最大公约数



题目描述

输入两个正整数 a 和 b ，用辗转相除法（欧几里得算法）求它们的最大公约数。

输入输出示例：

- 输入：24 18 → 输出：6
- 输入：17 5 → 输出：1

解题思路

辗转相除法原理： $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$

- **边界条件：** $b == 0$ 时， a 就是最大公约数
- **递归公式：** $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ （用余数替换，逐步缩小）

以 $\text{gcd}(24, 18)$ 为例：

步骤	a	b	a%b
1	24	18	6

例题 5：求最大公约数 —— 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int main() {
    cout << gcd(24, 18) << endl;    // 6
    return 0;
}
```

💡 这是著名的**欧几里得算法**，递归实现非常简洁。

例题 6：汉诺塔问题



题目描述

有 A、B、C 三根柱子，A 柱上有 n 个大小不同的盘子（大盘在下，小盘在上）。把所有盘子从 A 移到 C，移动规则：每次只能移一个盘子，且大盘不能放在小盘上。

输入输出示例：

- $n = 2$ ：A $\rightarrow$ B, A $\rightarrow$ C, B $\rightarrow$ C（共 3 步）
- $n = 3$ ：共 7 步

解题思路

分治思想：把“移 n 个盘子”拆成三步

1. 把上面 $n-1$ 个盘子从 A $\rightarrow$ B（借助 C）
2. 把最下面的 1 个盘子从 A $\rightarrow$ C
3. 把 $n-1$ 个盘子从 B $\rightarrow$ C（借助 A）

- **边界条件：** $n == 1$ 时，直接从 A $\rightarrow$ C
- **递归公式：** $\text{hanoi}(n) = \text{hanoi}(n-1) + \text{移1个} + \text{hanoi}(n-1)$

💡 最少步数 = $2^n - 1$



例题 6：汉诺塔问题 —— 参考代码



```
void hanoi(int n, char A, char B, char C) {  
    if (n == 1) {  
        cout << A << " → " << C << endl;  
        return;  
    }  
    hanoi(n - 1, A, C, B);    // 1. n-1 个 A→B (借 C)  
    cout << A << " → " << C << endl; // 2. 最下 A→C  
    hanoi(n - 1, B, A, C);    // 3. n-1 个 B→C (借 A)  
}
```

💡 这只是介绍，实际需要更多练习才能熟练。

递归的优缺点



优点:

- ✓ 代码简洁优雅, 解决"分形"问题非常自然
- ✓ 数学定义直接对应代码 (如阶乘、斐波那契)
- ✓ 适合"树"形结构 (汉诺塔、目录遍历)

缺点:

- ✗ 递归调用消耗栈空间, 层数太多会**栈溢出**
- ✗ 某些问题会**重复计算** (如斐波那契), 效率低
- ✗ 调试相对困难, 需要较强抽象思维

递归 vs 循环



特性	递归	循环
代码量	短小精悍	通常较长
性能	较慢（函数调用开销）	快
适用场景	树形、分治、自然递归	简单重复
栈空间	占用多	不额外占用

💡 **经验法则：**能用循环解决的不用递归；递归更直观时优先用递归。



常见错误与调试



错误类型	示例/说明	解决方法
缺少边界条件	递归永远不返回 → 栈溢出	必须明确"最小情况"返回值
边界条件错误	<code>if (n == 0)</code> 写成 <code>if (n == 1)</code>	画递归树确认边界
递归公式错误	<code>f(n-2)</code> 写成 <code>f(n+1)</code> → 越算越大	代入 <code>n=2,3</code> 验证
递归太深	<code>n = 100000</code> 递归阶乘 → 栈溢出	改为循环或加深栈
重复计算严重	斐波那契 <code>fib(40)</code> 极慢	加记忆化或改循环
忘记 return	边界情况没写 <code>return</code>	边界一定 return



知识小结



- ✓ **递归定义**：函数调用自身的编程技巧
- ✓ **两要素**：
 - **边界条件**：最小情况直接返回
 - **递归公式**：大问题转为小问题
- ✓ **执行过程**：先"递"层层调用，再"归"层层返回
- ✓ **经典示例**：阶乘、斐波那契、求和、数字之和、最大公约数、汉诺塔
- ✓ **注意事项**：必须保证递归能到达边界，否则会无限递归导致栈溢出
- ✓ **权衡**：递归代码简洁，但性能可能较差；能用循环的优先循环

下节课预告



下一课时：结构体（第19课时）

- 结构体的概念：自定义数据类型
- 结构体的定义与成员访问
- 结构体数组
- 结构体作为函数参数

"递归不是魔法——它只是把'大问题'拆成'同样的更小问题'。"

谢谢大家！

递归是函数式编程的核心思想，理解它就打开了更广阔编程世界的大门！

Q & A

